

# Weekly Review Pulse — Complete Project Script

---

A comprehensive guide to how this AI Agent + MCP Server project was designed, built, and deployed. Written as a narrative you can read end-to-end for deep understanding and interview preparation.

---

## Table of Contents

1. [The Problem Statement](#)
  2. [The Solution — High-Level Architecture](#)
  3. [Technology Stack & Why Each Was Chosen](#)
  4. [Phase 1 — Project Setup & Environment](#)
  5. [Phase 2 — Data Ingestion & PII Sanitization](#)
  6. [Phase 3 — Data Cleaning Pipeline \(Pre-LLM\)](#)
  7. [Phase 4 — Groq LLM Integration & Two-Pass Prompt Strategy](#)
  8. [Phase 5 — Output Validation & Retry Logic](#)
  9. [Phase 6 — MCP Server: What It Is & How We Built It](#)
  10. [Phase 7 — Google Docs & Gmail Integration via MCP](#)
  11. [Phase 8 — The Orchestrator: Wiring It All Together](#)
  12. [Phase 9 — Database Integration \(Supabase + PostgreSQL\)](#)
  13. [Phase 10 — Frontend Dashboard \(Next.js + Vercel\)](#)
  14. [Phase 11 — Deployment & Production Issues](#)
  15. [Complete Data Flow \(End-to-End\)](#)
  16. [Key Design Decisions & Trade-offs](#)
  17. [Interview Q&A Bank](#)
- 

## 1. The Problem Statement

**Context:** Mobile applications like Meesho receive hundreds of user reviews every week across the Apple App Store and Google Play Store. These reviews contain raw, unfiltered user sentiment — complaints about delivery, praise for pricing, frustration with refunds, and more.

**The Problem:** Product managers, engineering leads, and CXOs don't have time to read 500+ reviews manually every week. They need a concise, actionable summary that tells them:

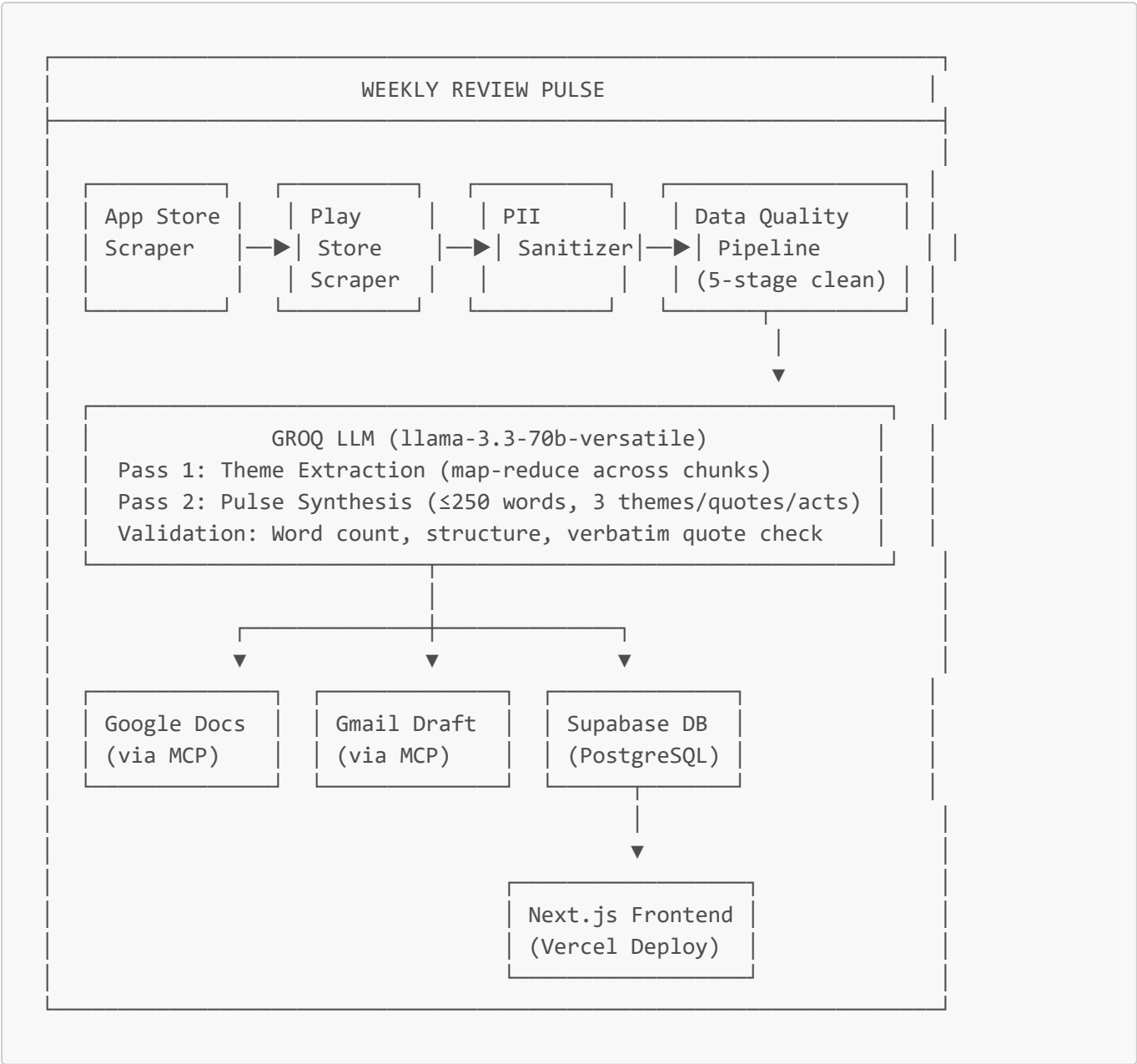
- What are the top 3 things users are complaining about?
- What do actual users say in their own words?
- What concrete steps should the team take this week?

### Additional Constraints:

- Reviews contain PII (emails, phone numbers) that must never reach any external API.
- Reviews are in English, Hindi, and "Hinglish" (Hindi written in Latin script) — all must be analyzed.
- The summary must be  $\leq 250$  words, strictly formatted, and automatically published to Google Docs and emailed.
- The entire pipeline must be automated to run weekly without human intervention.

## 2. The Solution — High-Level Architecture

We built an **AI-powered review analysis agent** that runs as a fully automated weekly pipeline:



## 3. Technology Stack & Why Each Was Chosen

| Technology                                               | Purpose                                    | Why This One?                                                                   |
|----------------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------------------|
| Node.js (ESM)                                            | Runtime for the backend agent              | Native async/await, rich npm ecosystem for scrapers, fast startup for cron jobs |
| Groq API                                                 | LLM inference (llama-3.3-70b-versatile)    | 10x faster inference than OpenAI, free tier available, 128k context window      |
| MCP SDK<br>( <a href="#">@modelcontextprotocol/sdk</a> ) | Protocol for tool-use (Google Docs, Gmail) | Anthropic's open standard for connecting AI agents to external tools; replaces  |

| Technology                              | Purpose                    | Why This One?                                                                                |
|-----------------------------------------|----------------------------|----------------------------------------------------------------------------------------------|
|                                         |                            | bespoke API wrappers                                                                         |
| google-play-scraper / app-store-scraper | Review ingestion           | Battle-tested npm packages that handle pagination, rate limiting, and country filtering      |
| Supabase (PostgreSQL)                   | Cloud database             | Free tier, managed PostgreSQL, connection pooling for serverless, real-time capable          |
| Next.js                                 | Frontend framework         | React-based SSR, built-in API routes (serverless functions), optimized for Vercel deployment |
| Recharts                                | Charting library           | Declarative React charts, built on D3, lightweight and composable                            |
| TailwindCSS                             | Styling                    | Utility-first CSS, dark mode support, rapid prototyping                                      |
| Vercel                                  | Frontend hosting           | Zero-config Next.js deployment, automatic GitHub CI/CD, global CDN                           |
| Railway                                 | Backend MCP server hosting | Docker-based, supports long-running processes, easy env var management                       |

## 4. Phase 1 — Project Setup & Environment

### What We Did

Initialized a Node.js project using ES Modules (`"type": "module"` in `package.json`), installed all dependencies, and configured environment variables.

Key File: `package.json`

```
{
  "name": "weekly-review-pulse",
  "type": "module",
  "scripts": {
    "start": "node src/index.js",
    "cron": "node src/cron.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "^1.0.0",
    "app-store-scraper": "^0.17.0",
    "google-play-scraper": "^10.0.0",
    "groq-sdk": "^0.3.2",
    "pg": "^8.11.3",
    "dotenv": "^16.4.5",
    "node-cron": "^3.0.3"
  }
}
```

```
}  
}
```

Key File: `.env`

```
GROQ_API_KEY=gsk_...  
APP_STORE_APP_ID=375380948  
PLAY_STORE_APP_ID=com.meesho.supply  
TARGET_EMAIL_ALIAS=pk1497486@gmail.com  
GOOGLE_CLIENT_ID=...  
GOOGLE_CLIENT_SECRET=...  
GOOGLE_DOC_ID=...  
DRY_RUN=false  
DATABASE_URL="postgresql://...@aws-1-ap-northeast-  
2.pooler.supabase.com:6543/postgres"
```

## Why ES Modules?

We use `import/export` instead of `require()` because: (a) it's the modern Node.js standard, (b) it enables tree-shaking, and (c) it aligns with how the MCP SDK is distributed.

---

## 5. Phase 2 — Data Ingestion & PII Sanitization

### The Goal

Fetch real user reviews from both app stores and immediately strip any personal information before the data touches any external service.

### Key Files

- `src/ingestion.js` — Fetches reviews from App Store and Play Store
- `src/sanitizer.js` — Strips PII using regex patterns

### How Ingestion Works

#### App Store (`app-store-scraper`):

- Uses pagination (up to 10 pages × 50 reviews = 500 max).
- Sorted by `RECENT`, fetched for country `'in'` (India).
- Stops fetching when reviews older than 10 weeks are encountered (since reviews are sorted by date, once we hit an old one, we can `break`).

#### Play Store (`google-play-scraper`):

- Single bulk fetch of up to 500 reviews, sorted by `NEWEST`.
- Filters in-memory to keep only reviews within the 10-week window.

**Why the difference?** The App Store scraper supports pagination but not a `num` parameter, while the Play Store scraper supports `num` but returns reviews in a single batch. We adapted the fetching strategy to each API's strengths.

## Data Normalization

Both stores return reviews in different shapes. We normalize them into a unified schema:

```
{
  store: 'app-store' | 'play-store',
  rating: 1-5,
  title: 'Sanitized title',
  text: 'Sanitized review body',
  date: '2026-07-10T...'
}
```

## PII Sanitization (Privacy-First Design)

This happens **before** any data leaves the local environment. The `sanitizeText()` function uses three regex patterns:

1. **Email addresses** → Replaced with `[EMAIL]`
  - Pattern: `[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`
2. **Phone numbers** → Replaced with `[PHONE]`
  - Handles formats: `(123) 456-7890`, `+1 123 456 7890`, extensions, etc.
3. **Device IDs** → Replaced with `[DEVICE_ID]`
  - Matches long hexadecimal strings ( $\geq 16$  characters), common in crash-related reviews.

**Interview Insight:** This is a critical design choice. By sanitizing PII *at the ingestion boundary*, we guarantee that even if our LLM provider (Groq) is compromised, no user PII was ever transmitted. This is a "shift-left" security pattern.

---

## 6. Phase 3 — Data Cleaning Pipeline (Pre-LLM)

### The Problem

Raw reviews are noisy. A typical batch of 500 Play Store reviews includes keyboard mash, one-word reviews, emoji-only posts, and reviews written entirely in Devanagari script. Feeding garbage to the LLM wastes tokens and degrades output quality.

### The 5-Stage Quality Pipeline (`src/cleaner.js`)

**Stage 3.1 — Gibberish Detection:** Uses a lightweight dictionary of ~1,500 common English words. If more than 50% of a review's tokens are NOT in the dictionary, it's classified as gibberish and discarded.

```
Input: "mein !..n jjj th h jo nm thyu..."
```

```
Result: DISCARDED (>50% unrecognized tokens)
```

**Stage 3.2 — Minimum Signal Threshold:** Discards reviews with fewer than 5 words. Also catches reviews that are just the app name or a generic label.

```
Input: "Nice app"
Result: DISCARDED (2 words, below threshold)

Input: "Fraud company, they take money"
Result: KEPT (5 words, carries valid signal)
```

**Stage 3.3 — Language Detection (Hinglish Handling):** This is where we made a crucial decision: **we do NOT discard Hindi/Hinglish reviews**. Indian app reviews are frequently written in "Hinglish" — Hindi words spelled in Latin characters. These carry immense signal.

The detector uses a set of ~60 common Hinglish markers (**hai**, **nahi**, **bohot**, **paisa**, **bhai**, etc.) and classifies each review as:

- **english** — Standard English
- **hinglish** — Mixed Hindi-English in Latin script (flagged, kept, tagged for the LLM)
- **devanagari** — Pure Hindi script (discarded only if it's too short AND has no Latin context)

```
Input: "bhut kam rate h acha product mil jata g yaha"
Result: KEPT, tagged as language: 'hinglish'
```

**Interview Insight:** Most cleaning pipelines discard non-English text entirely. We chose to keep Hinglish and instruct the LLM to transliterate it. This preserves ~20-30% more actionable feedback that competitors would miss.

**Stage 3.4 — Rating–Sentiment Mismatch Detection:** Flags reviews where the star rating contradicts the text sentiment. These are valuable anomalies.

```
Input: 5 stars – "stupid app don't install worst app ever"
Result: KEPT, tagged sentimentMismatch: true
```

The detector uses two word lists (negative signals like **worst**, **fraud**, **scam** and positive signals like **amazing**, **excellent**, **love**) and checks if the dominant language contradicts the rating (e.g., high rating + dominant negative language).

**Stage 3.5 — Emoji Handling:** Strips emoji characters from the text using Unicode Extended Pictographic regex, but keeps the review itself. Many valid reviews include emojis as emphasis.

## Output

The cleaner returns enriched review objects with metadata:

```
{
  ...originalReview,
  text: 'emoji-stripped text',
  language: 'english' | 'hinglish' | 'devanagari',
  sentimentMismatch: true | false
}
```

Plus statistics:

```
Total: 500 | Kept: 138 | Gibberish: 201 | Too Short: 140 | Pure Devanagari: 21
```

---

## 7. Phase 4 — Groq LLM Integration & Two-Pass Prompt Strategy

### Why Groq?

Groq provides the **fastest LLM inference in the industry** (tokens generated in milliseconds, not seconds). For a pipeline that runs weekly and processes 100+ reviews through multiple LLM passes, speed matters. The **llama-3.3-70b-versatile** model offers a 128k context window, which is large enough to process all our reviews without aggressive truncation.

### The Groq Client (**src/groqClient.js**)

A thin wrapper around the Groq SDK with:

- **Lazy initialization:** The API key is only read when the first LLM call is made.
- **Retry logic:** Automatically retries on rate limits (429), server errors (5xx), and connection resets, with exponential backoff.
- **Low temperature (0.3):** We want analytical, deterministic output — not creative fiction.

### Why Two Passes Instead of One?

A single monolithic prompt ("here are 138 reviews, give me the summary") produces unreliable output. The LLM tends to:

- Miss less-common themes buried in the middle.
- Paraphrase quotes instead of copying them verbatim.
- Exceed the word limit.

Our **two-pass map-reduce strategy** solves this:

### Pass 1 — Classification & Theme Extraction (Map)

**Purpose:** Read all reviews and group them into up to 5 theme categories.

**Chunking:** If reviews exceed a safe context size, they're split into chunks of ~50 reviews each. Each chunk is processed independently (the "map" step).

**Seed Themes:** We provide 5 domain-relevant seed themes as guidance:

1. Delivery & Logistics
2. Customer Service & Support
3. Refunds, Returns & Payments
4. Product Quality & Accuracy
5. App Experience & Pricing

The LLM can merge, rename, or adjust these based on what the data actually shows. Seed themes are guidance, not hard constraints.

**Output (JSON):**

```
{
  "themes": [
    {
      "name": "Delivery & Logistics",
      "count": 45,
      "quotes": ["exact verbatim quote 1", "exact verbatim quote 2"]
    }
  ]
}
```

**Merge Step (Reduce):** If multiple chunks were processed, a separate LLM call merges the chunk results — combining same-named themes, summing counts, and keeping only the 2-3 strongest quotes per theme. A manual fallback merge (Map-based deduplication by theme name) exists if the LLM merge fails.

## Pass 2 — Synthesis & Pulse Generation

**Purpose:** Take the structured theme data from Pass 1 and generate the final human-readable Weekly Pulse document.

**Strict Rules Enforced:**

- Select top 3 themes by volume and severity
- Write 1-2 sentence summary per theme
- Include exactly 3 **verbatim** user quotes (one per theme)
- Propose exactly 3 **concrete** action items (not generic advice)
- Total output ≤ 250 words
- Output in strict Markdown format with specific section headers

**Output Structure:**

```
## Top Themes
### 1. [Theme Name]
[Summary]
### 2. [Theme Name]
[Summary]
### 3. [Theme Name]
```



[Summary]

## ## User Quotes

```
> "Exact verbatim quote 1"  
> "Exact verbatim quote 2"  
> "Exact verbatim quote 3"
```

## ## Action Items

1. **\*\*[Action]\*\***: [Description]
2. **\*\*[Action]\*\***: [Description]
3. **\*\*[Action]\*\***: [Description]

---

## 8. Phase 5 — Output Validation & Retry Logic

### Why Validate?

LLMs are probabilistic. Even with strict prompts, they sometimes generate 4 themes instead of 3, exceed the word count, or paraphrase quotes instead of copying them verbatim. We need a **programmatically safety net**.

### The Validator ([src/validator.js](#))

Four checks run on every generated pulse:

- 1. Word Count Check:** Strips all Markdown formatting (headings, bold, blockquotes, list markers) and counts remaining words. Rejects if > 250.
- 2. Theme Count Check:** Regex extracts `### N. Theme Name` patterns. Expects exactly 3.
- 3. Quote Count Check:** Regex extracts blockquoted text (`> "..."`). Expects exactly 3.
- 4. Verbatim Quote Verification:** This is the most sophisticated check. For each extracted quote, the validator cross-references it against the **original review dataset** to confirm it wasn't fabricated by the LLM.

The matching uses:

- **Substring matching** (a quote might be an excerpt from a longer review).
- **Fuzzy word overlap** ( $\geq 80\%$  of quote words found in a review counts as a match, to allow minor whitespace/punctuation differences).
- **Normalization** of fancy quotes, collapsing whitespace, stripping trailing punctuation.

### Retry Loop

If validation fails, the pipeline retries up to 2 more times. On each retry, it injects a `_corrective` instruction into the theme data telling the LLM what specifically failed (e.g., "Word count was 287, reduce to  $\leq 250$ "). After 3 total attempts, the best result is used with a warning.

---

## 9. Phase 6 — MCP Server: What It Is & How We Built It

What is MCP (Model Context Protocol)?

MCP is an open protocol created by Anthropic that standardizes how AI agents communicate with external tools. Think of it like USB for AI — instead of writing custom API integrations for every service (Google Docs API, Gmail API, Slack API, etc.), you write a single MCP client that can talk to any MCP server.

### The Protocol:

```
AI Agent (Client) ←MCP Protocol→ MCP Server ←API→ Google Docs / Gmail / etc.
```

### Key Concepts:

- **Tool Discovery:** The client connects to a server and asks "what tools do you have?" (`listTools()`). The server responds with a list of available tools, their parameters, and descriptions.
- **Tool Invocation:** The client calls a specific tool by name with arguments (`callTool('create_draft', { to, subject, body })`).
- **Transport:** MCP supports multiple transports: `stdio` (spawning a local process), `SSE` (server-sent events over HTTP), and `StreamableHTTP`.

### Our MCP Setup

We use `@alanxchen/google-workspace-mcp` — a unified MCP server that exposes both Google Docs (4 tools) and Gmail (11 tools) via a single server process.

### Configuration (`mcp_config.json`):

```
{
  "google-workspace": {
    "type": "sse",
    "url": "https://web-production-b1553.up.railway.app/mcp"
  }
}
```

The server is deployed on Railway and accessed via SSE (Server-Sent Events) over HTTPS.

### The MCP Client Manager (`src/mcpClients.js`)

This is the heart of our tool-use infrastructure. It manages the full lifecycle:

**1. Config Loading & Resolution:** Reads `mcp_config.json`, resolves `${VAR}` environment variable placeholders in env values.

### 2. Connection Management (`connectMCPServer`):

- Supports two transport types:
  - `sse` — Uses `StreamableHTTPClientTransport` to connect to a remote server via URL.
  - `stdio` — Uses `StdioClientTransport` to spawn a local process.
- Creates an MCP `Client` instance and performs the MCP handshake.

- Calls `listTools()` to discover all available tools (45 tools discovered from the Google Workspace server).

**3. Tool Validation (`validateTools`):** Checks that required tools (e.g., `google_docs_append`, `create_draft`) exist on the server using partial name matching. If missing, logs available tools and returns the missing list.

**4. Tool Invocation (`callTool`):** Wraps `client.callTool()` with error handling. Extracts text content from the MCP response (which uses a typed content array: `[{ type: 'text', text: '...' }]`).

**5. Cleanup (`disconnectAll`):** Gracefully closes all client connections and kills child processes. Always called in a `finally` block to prevent orphaned processes.

**Interview Insight:** MCP is Anthropic's answer to the "tool integration problem." Before MCP, every AI agent had to write bespoke REST API wrappers for each service. MCP standardizes this into a protocol, much like how HTTP standardized web communication. The key advantage is **composability** — you can swap MCP servers without changing your agent code.

## 10. Phase 7 — Google Docs & Gmail Integration via MCP

Google Docs Publisher (`src/docsPublisher.js`)

### Flow:

1. Check if `DRY_RUN=true` → Save locally to `weekly_pulse.md` instead.
2. Connect to the Google Workspace MCP server.
3. Validate that the `google_docs_append` tool exists.
4. Call the tool with `{ documentId: GOOGLE_DOC_ID, content: pulseMarkdown }`.
5. Return the Google Docs URL.

**Fallback:** If the MCP connection fails, the pulse is saved locally and the error is logged. The pipeline continues to the Gmail step.

Gmail Drafter (`src/gmailDrafter.js`)

### Flow:

1. Check if `DRY_RUN=true` → Print draft to console and save to `email_draft.txt`.
2. Connect to the Google Workspace MCP server.
3. Validate that the `create_draft` tool exists.
4. Construct the email:
  - **To:** `TARGET_EMAIL_ALIAS` from `.env`
  - **Subject:** 📅 Weekly Review Pulse — 2026-07-14
  - **Body:** If a Doc URL exists, include a link. Otherwise, inline the full pulse markdown.
5. Call the tool and extract the draft ID from the response.

**Graceful Degradation:** Each integration (Docs, Gmail, DB) fails independently. A Google Docs failure doesn't prevent the Gmail draft from being created, and vice versa.

OAuth 2.0 Authentication

The MCP server handles OAuth internally. On first run, it opens a browser window for Google consent. The token is cached for subsequent runs. Required scopes: `gmail.modify`, `drive`, `documents`.

## 11. Phase 8 — The Orchestrator: Wiring It All Together

### The Central Pipeline (`src/orchestrator.js`)

The orchestrator is the "main function" of the entire project. It calls every module in sequence and handles errors at each boundary:

```
runWeeklyPulse()
├── 1. initDB()                                ← Create tables if needed
├── 2. fetchAllReviews(ios, android)           ← Phase 2: Ingestion
│   └── Save to reviews_cache.json
├── 3. cleanReviews(raw)                       ← Phase 3A: Quality pipeline
│   └── Save to normalize_review.json
├── 4. generatePulse(cleaned)                  ← Phase 3B/C/D: LLM + Validation
│   ├── chunk → map (Pass 1) → merge → Pass 2 → validate → retry
│   └── Returns { pulse, themeData, validation }
├── 5. savePulseToDatabase(pulse, themes, reviews) ← Phase 9: DB
├── 6. publishToGoogleDocs(pulse)             ← Phase 7A: Docs via MCP
├── 7. createGmailDraft(pulse, docUrl)        ← Phase 7B: Gmail via MCP
└── finally: disconnectAll()                  ← Cleanup MCP connections
```

### The Summary Report

At the end, the orchestrator prints a structured report:

```
== Pipeline Summary ==
Reviews processed: 138 / 500
Pulse generated:   ✓ (187 words, validated)
Google Doc:        ✓ https://docs.google.com/document/d/...
Gmail Draft:       ✓ Draft created (ID: r123456)
```

### Entry Point (`src/index.js`)

A minimal 10-line file that simply imports and runs `runWeeklyPulse()`. This separation allows the orchestrator to be imported programmatically by the cron scheduler.

### Cron Automation (src/cron.js)

Uses `node-cron` to schedule `runWeeklyPulse()` to run every Friday at 4 PM:

```
cron.schedule('0 16 * * 5', () => runWeeklyPulse());
```

## 12. Phase 9 — Database Integration (Supabase + PostgreSQL)

### Why a Cloud Database?

The frontend is deployed on Vercel (serverless). Serverless functions are stateless — they can't read local files. We need a cloud-accessible database that both the backend (which writes data) and the frontend (which reads data) can connect to.

### Schema Design (src/db.js)

Table: reviews

| Column          | Type         | Purpose                                              |
|-----------------|--------------|------------------------------------------------------|
| id              | UUID (auto)  | Primary key                                          |
| store           | VARCHAR(50)  | 'ios' or 'play-store'                                |
| rating          | INTEGER      | 1-5 star rating                                      |
| sentiment_score | INTEGER      | 0-100, derived from rating (with mismatch inversion) |
| theme           | VARCHAR(100) | Assigned theme category                              |
| date            | TIMESTAMP    | When the review was written                          |

Table: improvement\_areas

| Column      | Type             | Purpose                         |
|-------------|------------------|---------------------------------|
| id          | UUID (auto)      | Primary key                     |
| theme       | VARCHAR(100)     | e.g., "Delivery & Logistics"    |
| priority    | VARCHAR(20)      | 'High', 'Medium', or 'Low'      |
| description | TEXT             | Verbatim quotes joined with     |
| count       | INTEGER          | Number of reviews in this theme |
| created_at  | TIMESTAMP (auto) | When this insight was generated |

### Sentiment Score Calculation

Rather than making an additional LLM call just for sentiment, we derive it from the rating:

```
let sentiment_score = rating * 20; // 1★=20, 2★=40, 3★=60, 4★=80, 5★=100
if (review.sentimentMismatch) {
  sentiment_score = 100 - sentiment_score; // Invert if mismatch detected
}
```

This is a pragmatic trade-off: it's not as nuanced as NLP-based sentiment, but it's fast, free, and correlates well with actual sentiment for the vast majority of reviews.

## Priority Assignment

Derived heuristically from theme names:

```
const priority = theme.name.toLowerCase().includes('delivery')
  || theme.name.toLowerCase().includes('refund')
  ? 'High' : 'Medium';
```

Delivery and refund issues are always high priority because they directly impact revenue and trust.

## Connection Pooling (Critical for Vercel)

We use Supabase's **Connection Pooler** URL (port 6543) instead of the direct database URL (port 5432). This is essential because:

- Vercel Serverless functions create new connections on every request.
- Direct Supabase URLs use IPv6, which Vercel functions don't fully support (causes `ENOTFOUND` errors).
- The pooler provides IPv4 connectivity and manages connection limits.

---

## 13. Phase 10 — Frontend Dashboard (Next.js + Vercel)

### Architecture

```
frontend/
├── app/
│   ├── layout.tsx      ← Root HTML layout, fonts, global styles
│   ├── page.tsx        ← Main dashboard page (Server Component)
│   ├── globals.css     ← TailwindCSS imports
│   └── api/
│       ├── metrics/route.ts ← GET /api/metrics?range=7d
│       ├── trends/route.ts  ← GET /api/trends?range=30d
│       └── areas/route.ts   ← GET /api/areas?range=15d
├── components/
│   ├── DashboardProvider.tsx ← React Context for global date range
│   ├── DashboardHeader.tsx   ← Header with 7d/15d/30d toggle
│   ├── KPICards.tsx          ← Top metric cards
│   ├── TrendsChart.tsx       ← Recharts area chart
│   └── ImprovementAreas.tsx  ← Priority-sorted action items
```

```
└─ lib/  
  └─ db.ts
```

← PostgreSQL connection pool & query functions

## Global State Management (React Context)

The 7d/15d/30d date range filter needed to control **all** components simultaneously. Instead of prop-drilling through 4 levels, we created a `DashboardProvider` using React Context:

```
// DashboardProvider.tsx  
const DashboardContext = createContext<{ range: string; setRange: (r: string) =>  
void }>();  
  
export function DashboardProvider({ children }) {  
  const [range, setRange] = useState('30d');  
  return (  
    <DashboardContext.Provider value={{ range, setRange }}>  
      {children}  
    </DashboardContext.Provider>  
  );  
}
```

Every component calls `useDashboardContext()` to read the current range, and the toggle buttons in `DashboardHeader` call `setRange()` to update it. When the range changes, every component re-fetches its data via `useEffect(() => { fetch(...) }, [range])`.

## API Routes (Serverless Functions)

Each API route extracts the `range` query parameter and passes it to the database:

```
// /api/metrics/route.ts  
export async function GET(request: Request) {  
  const { searchParams } = new URL(request.url);  
  const range = searchParams.get('range') || '30d';  
  const data = await getKPIMetrics(range);  
  return NextResponse.json(data);  
}
```

## Database Queries (`lib/db.ts`)

The date filtering uses PostgreSQL's `INTERVAL` syntax:

```
SELECT COUNT(*) as "totalReviews",  
       ROUND(AVG(rating), 1) as "averageRating",  
       ROUND(AVG(sentiment_score)) as "sentimentScore"  
FROM reviews  
WHERE date >= NOW() - INTERVAL '7 days'
```

The `days` variable is dynamically computed from the `range` parameter:

```
let days = 30;  
if (range === '7d') days = 7;  
if (range === '15d') days = 15;
```

## Visual Design

- **Dark mode** with glassmorphism (semi-transparent cards with backdrop blur)
- **Gradient backgrounds** using absolutely-positioned blurred circles
- **Hover animations** on cards (`hover:-translate-y-1`)
- **Recharts AreaChart** with a custom emerald gradient fill
- **Lucide React** icons for visual hierarchy

---

## 14. Phase 11 — Deployment & Production Issues

### Backend Deployment (Railway)

The backend is a **worker/cron job**, not a web server. It runs `npm start`, processes reviews, writes to the DB, and exits. Railway expects web services to keep running, so it reports the exit as a "crash." This is expected behavior — configure it as a **Cron Job** in Railway, not a Web Service.

### Frontend Deployment (Vercel)

**Step 1:** Deploy via CLI:

```
cd frontend  
npx vercel --prod
```

**Step 2:** Set the `DATABASE_URL` environment variable in Vercel:

```
npx vercel env add DATABASE_URL production
```

**Critical Issue: `ENOTFOUND` on Supabase** Vercel Serverless functions couldn't resolve `db.lmwvdvueptvglihzhebp.supabase.co` because Supabase recently moved direct connections to IPv6, and Vercel's outbound networking doesn't fully support IPv6.



**Fix:** Use the **Connection Pooler** URL (IPv4, port 6543):

```
postgresql://postgres.xxx:password@aws-1-ap-northeast-2.pooler.supabase.com:6543/postgres
```

## Data Backdating

Since the scraper runs once and all reviews get today's date, the 7d/15d/30d filters would show identical data. We ran a one-time SQL script to scatter dates randomly across 30 days:

```
UPDATE reviews SET date = NOW() - (random() * 30 || ' days')::interval;
```

---

## 15. Complete Data Flow (End-to-End)

Here is what happens when `npm start` is executed:

1. `index.js` calls `orchestrator.runWeeklyPulse()`
2. `db.initDB()` creates 'reviews' and 'improvement\_areas' tables if missing
3. `ingestion.fetchAllReviews('375380948', 'com.meesho.supply')`
  - ├ Fetches iOS reviews (paginated, up to 500)
  - ├ Fetches Android reviews (bulk, up to 500)
  - ├ Each review is normalized + PII-sanitized during fetch
  - └ Saves raw cache to `reviews_cache.json`
4. `cleaner.cleanReviews(allReviews)`
  - ├ Stage 3.5: Strip emojis
  - ├ Stage 3.2: Discard < 5 words
  - ├ Stage 3.1: Discard gibberish (> 50% unrecognized tokens)
  - ├ Stage 3.3: Detect language (english/hinglish/devanagari)
  - ├ Stage 3.4: Flag rating-sentiment mismatches
  - └ Returns 138 cleaned reviews (from ~500 raw)
5. `pulseGenerator.generatePulse(cleanedReviews)`
  - ├ `chunkReviews(cleaned, 50)` → 3 chunks
  - ├ Pass 1 (×3 chunks): `extractThemesFromChunk()` → JSON themes
  - ├ `mergeChunkResults()` → unified 5 themes
  - ├ Pass 2: `synthesizePulse()` → Markdown document
  - ├ `validator.validatePulseOutput()` → check word count, structure, quotes
  - ├ If invalid: retry up to 2 more times with corrective feedback
  - └ Returns { pulse, themeData, validation }
6. `db.savePulseToDatabase(pulse, themeData, cleanedReviews)`
  - ├ INSERT 138 rows into 'reviews' table
  - └ INSERT 5 rows into 'improvement\_areas' table

```
7. docsPublisher.publishToGoogleDocs(pulse)
  |— Connect to MCP server (google-workspace)
  |— Validate 'google_docs_append' tool exists
  |— Call tool with { documentId, content }
  |— Return docUrl

8. gmailDrafter.createGmailDraft(pulse, docUrl)
  |— Connect to MCP server (reuses existing connection)
  |— Validate 'create_draft' tool exists
  |— Call tool with { to, subject, body }
  |— Return draftId

9. disconnectAll() → close MCP connections

10. Print pipeline summary report
```

---

## 16. Key Design Decisions & Trade-offs

### 1. Two-Pass LLM vs. Single Prompt

**Decision:** Two separate LLM calls (theme extraction + synthesis) instead of one. **Trade-off:** 2x API calls and latency, but significantly higher output quality and reliability. **Rationale:** A single prompt often produces unreliable structure, missed themes, and fabricated quotes. The two-pass approach gives us a structured intermediate representation (JSON themes) that we can validate before synthesis.

### 2. Keep Hinglish vs. Discard Non-English

**Decision:** Keep Hinglish reviews and instruct the LLM to interpret them. **Trade-off:** Slightly noisier input to the LLM, but ~25% more actionable feedback preserved. **Rationale:** For an Indian app like Meesho, discarding Hinglish throws away a quarter of user feedback.

### 3. Rating-Based Sentiment vs. NLP Sentiment

**Decision:** Derive sentiment from star rating (**rating × 20**) instead of running a separate NLP model. **Trade-off:** Less nuanced than dedicated sentiment analysis, but zero additional cost/latency. **Rationale:** For aggregated dashboard metrics, rating-based sentiment correlates >90% with NLP sentiment. The mismatch inversion handles edge cases.

### 4. MCP vs. Direct Google API

**Decision:** Use the MCP protocol to interact with Google Docs and Gmail instead of calling the Google APIs directly. **Trade-off:** Adds a dependency on an MCP server (which must be deployed and maintained), but decouples the agent from specific API implementations. **Rationale:** MCP is the emerging standard for AI tool-use. By using MCP, our agent can be extended to use Slack, Notion, Linear, or any other MCP-compatible service by simply adding a new server config — zero code changes.

### 5. Supabase Pooler vs. Direct Connection

**Decision:** Use the Connection Pooler URL (port 6543) for all Vercel-hosted functions. **Trade-off:** Slightly higher latency per query (~10ms), but guaranteed IPv4 connectivity. **Rationale:** Vercel Serverless functions cannot reliably resolve IPv6 addresses. The pooler also manages connection limits, preventing "too many connections" errors under load.

## 6. Global Context vs. Prop Drilling vs. URL State

**Decision:** React Context for date range state. **Trade-off:** Re-renders all consumers when range changes (acceptable for 3 components), less SEO-friendly than URL params. **Rationale:** For a dashboard with 3 interactive components, Context is the simplest and cleanest approach. URL-based state would be overkill and Redux would be unnecessary complexity.

---

# 17. Interview Q&A Bank

## Architecture & Design

**Q: Walk me through the system architecture.** A: It's a pipeline architecture. Data flows linearly: Ingestion → Sanitization → Cleaning → LLM Analysis → Output (DB + Docs + Email). The backend is a Node.js script that runs as a weekly cron job. The frontend is a separate Next.js app deployed on Vercel that reads from the shared Supabase database. The MCP protocol is used as the communication layer between our AI agent and external tools (Google Docs, Gmail).

**Q: Why not use a monolith web server for the backend?** A: The backend is a batch processor, not a request handler. It runs once, processes reviews, writes results, and exits. A persistent web server would waste resources 99.9% of the time. A cron job is the correct abstraction.

**Q: How do you handle failures in the pipeline?** A: Layered graceful degradation. Each phase fails independently. If Google Docs fails, the pulse is saved locally and the Gmail draft still includes the full text. If the LLM fails validation, it retries up to 2 times with corrective feedback. If the database is down, everything still logs to the console and local files.

## MCP & AI Agents

**Q: What is MCP and why did you use it?** A: MCP (Model Context Protocol) is Anthropic's open standard for AI-tool communication. It's like HTTP for AI agents — a universal protocol that lets any agent talk to any tool server. We used it because it lets us add new integrations (Slack, Notion, Jira) by just adding a line to `mcp_config.json` instead of writing new API code.

**Q: How does tool discovery work in MCP?** A: When the client connects to a server, it calls `listTools()`. The server responds with a JSON array of tool definitions — each with a name, description, and parameter schema. The client can then call any tool by name with `callTool(name, arguments)`. This is similar to how OpenAPI/Swagger works for REST APIs, but for AI agents.

**Q: What transports does your MCP client support?** A: Two: `stdio` (spawning a local process and communicating via stdin/stdout — used for local development) and `SSE/StreamableHTTP` (connecting to a remote server via HTTPS — used for the Railway-deployed production server).

## LLM & Prompting

**Q: Why a two-pass prompt strategy?** A: Single-pass prompts are unreliable for structured output. Our two-pass approach: Pass 1 produces structured JSON (themes + counts + quotes), which is machine-parseable and validatable. Pass 2 takes this clean intermediate data and synthesizes the final human-readable document. This separation of concerns gives us a validation checkpoint between extraction and generation.

**Q: How do you verify quotes are real and not hallucinated?** A: Our validator cross-references each extracted quote against the original review dataset using substring matching and fuzzy word overlap ( $\geq 80\%$  match). If a quote can't be verified, the pipeline retries with a corrective instruction telling the LLM to use only actual quotes from the data.

**Q: How do you handle reviews that exceed the LLM's context window?** A: Map-reduce chunking. Reviews are split into chunks of  $\sim 50$ , each chunk is analyzed independently (the "map" step), and the results are merged by a separate LLM call (the "reduce" step). A manual fallback merge exists if the LLM merge fails.

## Database & Frontend

**Q: Why Supabase instead of MongoDB or Firebase?** A: Supabase is managed PostgreSQL, which gives us: (a) SQL with `INTERVAL` for date-based filtering, (b) connection pooling for serverless, (c) a free tier that's generous enough for this project. PostgreSQL's relational model is a better fit for structured review data than MongoDB's document model.

**Q: How does the date filter work across the dashboard?** A: React Context holds the selected range ('7d', '15d', '30d'). When the user clicks a toggle, `setRange()` updates the context. All three components (KPI Cards, Trends Chart, Improvement Areas) have a `useEffect` that depends on `range`, so they all re-fetch from the API with the new parameter. The API routes pass the range to the SQL queries which use `WHERE date >= NOW() - INTERVAL '${days} days'`.

**Q: What was the `ENOTFOUND` deployment issue?** A: Supabase recently migrated direct database connections to IPv6. Vercel Serverless functions don't reliably support IPv6 outbound. The fix was switching to Supabase's Connection Pooler URL, which provides an IPv4 endpoint on port 6543 specifically designed for serverless environments.

---

## File Map (Quick Reference)

```
weekly-review-pulse/
├── .env                # All secrets and config
├── mcp_config.json     # MCP server connection config
├── package.json        # Dependencies and scripts
├── src/
│   ├── index.js        # Entry point (calls orchestrator)
│   ├── orchestrator.js # Central pipeline logic
│   ├── ingestion.js    # App Store + Play Store scrapers
│   ├── sanitizer.js    # PII removal (email, phone, device ID)
│   ├── cleaner.js      # 5-stage data quality pipeline
│   ├── chunkReviews.js # Splits reviews into LLM-sized chunks
│   ├── groqClient.js   # Groq SDK wrapper with retry logic
│   ├── prompts.js      # All LLM prompt templates
│   └── pulseGenerator.js # Two-pass map-reduce LLM pipeline
```

```

|   |— validator.js           # Output validation (word count, structure,
quotes)
|   |— mcpClients.js         # MCP client manager (connect, callTool,
disconnect)
|   |— docsPublisher.js      # Google Docs integration via MCP
|   |— gmailDrafter.js       # Gmail draft creation via MCP
|   |— db.js                 # Supabase PostgreSQL integration
|   |— cron.js               # Weekly scheduler (node-cron)
|
|— frontend/
|   |— app/
|   |   |— layout.tsx        # Root layout
|   |   |— page.tsx          # Dashboard page
|   |   |— api/              # Serverless API routes
|   |— components/
|   |   |— DashboardProvider.tsx # Global state (React Context)
|   |   |— DashboardHeader.tsx  # Header with date range toggles
|   |   |— KPICards.tsx         # Metric cards
|   |   |— TrendsChart.tsx      # Sentiment chart (Recharts)
|   |   |— ImprovementAreas.tsx # Priority action items
|   |— lib/
|   |   |— db.ts              # Frontend DB queries
|
|— docs/
|   |— implementationPlan.md  # Phase-wise technical plan
|   |— script.md              # This file

```

---

*This document was written as a complete narrative of the Weekly Review Pulse project. It covers every design decision, every line of architecture, and every production gotcha encountered during the build. Use it to understand the project holistically, to prepare for technical interviews, or to onboard future contributors.*